SC4052 Cloud Computing Course Project Topic

Topic 6: Debate Preparation coach on Nemobot

Prepared by:

Dominic Lai Meng Xuan

U2221785D

Contents

1 Introduction **3**

2 Literature Review **3**

2.1 Prompt Decomposition: Decomposed Prompting (DECOMP) 3

2.2 Prompt Differentiation: TextGrad 4

2.3 Combining Decomposition and Differentiation 5

3 Problem Statement **5**

4 Solution Design **6**

4.1 System Architecture 6

4.2 Phase 1: Decomposition Pipeline 7

4.3 Phase 2: TextGrad Differentiation Loop 8

4.4 Phase 3: Synthesis 9

4.5 Implementation on Nemobot 9

4.6 Design Decisions and Limitations 10

5 Illustrative Examples **10**

5.1 Test Topic 10

5.2 Phase 1: Decomposition Output (LLM-Driven) 11

5.2.1 Decomposer Plan 11

5.2.2 Stage 1 — Argument Extraction (4.7s) 11

5.2.3 Stage 2 — Counterargument Generation (6.3s) 12

5.2.4 Stage 3 — Rebuttal Crafting (9.5s) 12

5.3 Phase 2: TextGrad Differentiation Loop 12

5.3.1 Score Progression Across Rounds 12

5.3.2 Textual Gradient (Round 1) 13

5.3.3 Before/After Comparison: Argument 1 13

5.4 Phase 3: Final Debate Brief 14

5.5 Pipeline Performance Summary 16

6 Conclusion **16**

6.1 Key Findings 16
6.2 Future Work 17

7 Appendix: LLM Prompts + code **17**

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities in natural language reasoning, generation, and analysis. However, when tasked with complex, multi-faceted problems, a single monolithic prompt often produces outputs that are shallow, internally inconsistent, or missing critical components. Debate preparation is a representative example of such a task: it requires argument construction, adversarial thinking, evidence gathering, self-evaluation, and coherent synthesis, all within a single workflow.

This project explores how two recent advances in prompt engineering can be combined to address these limitations:

1. **Prompt Decomposition** (Khot et al., 2023) — breaking a complex task into a pipeline of smaller, focused sub-prompts, where each sub-prompt handles one well-defined sub-task and passes its output to the next stage.

2. **Prompt Differentiation / TextGrad** (Yuksekgonul et al., 2024) — using an evaluator LLM to score outputs and produce natural language "gradients" (specific improvement instructions), which are fed back into earlier stages to iteratively refine quality.

We implement these techniques as a **Debate Preparation Coach** on Nemobot, an agentic AI SaaS platform that supports tool-augmented generation and programmable prompts. The system accepts a debate topic and position, orchestrates five LLM function calls in sequence (decomposition), evaluates the combined output and iteratively refines weak stages using textual gradients (differentiation), and produces a final debate brief.

2 Literature Review

2.1 Prompt Decomposition: Decomposed Prompting (DECOMP)

Khot et al. (2023) introduced *Decomposed Prompting* (DECOMP) , proposing a modular approach to solving complex tasks with LLMs. The core insight is that rather than relying on a single prompt to handle an entire multi-step problem, the task should be broken into sub-tasks, each handled by a dedicated "sub-prompt" or "handler." These handlers can be few-shot prompted LLMs, symbolic functions, or even calls to external tools.

DECOMP has the following key advantages:

- **Modularity:** Each sub-task is handled by a specialised prompt that can be independently optimised and tested.

- **Composability:** A decomposer prompt generates a sequence of sub-task calls, and the outputs are composed to produce the final answer.
- **Tool integration:** Sub-tasks can delegate to external tools (retrieval, computation), enabling the LLM to operate beyond its native capabilities.
- **Improved performance:** On benchmarks requiring multi-step reasoning (e.g., multi-hop QA, long-context tasks), DECOMP outperforms chain-of-thought prompting by reducing the cognitive burden on any single prompt.

In our system, each stage of debate preparation (argument extraction, counterargument generation, etc.) is a separate Nemobot LLM function.

2.2 Prompt Differentiation: TextGrad

Yuksekgonul et al. (2024) introduced *TextGrad*, drawing an analogy between numerical gradient descent in deep learning and iterative text-based feedback in LLM systems. In traditional deep learning, a loss function computes a scalar error, and gradients flow backward through the network to update weights. TextGrad replaces each component of this process with natural language:

- **Variables** are text strings (prompts, outputs, code, etc.) that can be "optimised."
- **Loss function** is an LLM call that evaluates the quality of a variable and produces a natural language critique.
- **Textual gradients** are the specific, actionable improvement instructions produced by the loss function.
- **Gradient descent step** involves feeding the textual gradients back into the original prompt and re-generating the output.

The formal analogy is:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t) \qquad \text{(numerical gradient descent)} \tag{1}$$

$$\text{prompt}_{t+1} = \text{LLM}(\text{prompt}_t, \text{gradient}_t) \qquad \text{(TextGrad)} \tag{2}$$

where the "learning rate" η is implicitly controlled by the strength and specificity of the textual gradient, and the "update step" is a prompted LLM call that incorporates the feedback.

TextGrad demonstrated improvements across diverse tasks including code optimisation, reasoning, and scientific question answering. The key advantage is generality: any system where outputs can be evaluated in natural language can be optimised using TextGrad, without requiring numerical loss functions or differentiable architectures.

2.3 Combining Decomposition and Differentiation

Our work combines these two techniques. Decomposition provides the *pipeline structure* to break the overarching task of debate preparation into modular stages. Differentiation provides the *quality improvement mechanism*, iteratively refining each stage's output based on evaluative feedback.

3 Problem Statement

Debate preparation is a complex, multi-faceted task that requires:

1. **Argument construction:** Identifying the strongest claims supporting a given position, each grounded in evidence and logical reasoning.
2. **Adversarial analysis:** Anticipating the counterarguments an opponent might raise.
3. **Rebuttal preparation:** Crafting responses to each anticipated counterargument that acknowledge the opponent's point while redirecting to stronger ground.
4. **Self-evaluation:** Critically assessing the overall quality of the preparation across multiple dimensions (coherence, specificity, adversarial strength, completeness, persuasiveness).
5. **Synthesis:** Combining all materials into a coherent, podium-ready document.

When approached with a single monolithic prompt, LLMs typically produce outputs that are:

- **Shallow:** Arguments lack specific evidence or data points.
- **Internally inconsistent:** Rebuttals may contradict earlier arguments.
- **One-pass:** The output is never self-evaluated or refined, so weaknesses persist.

- **Structurally incomplete:** Critical sections (vulnerabilities, preparation notes) are often missing.

Our goal is to design a system that:

1. Decomposes debate preparation into specialised sub-tasks (prompt decomposition).
2. Iteratively refines the output of each sub-task using evaluative feedback (prompt differentiation).
3. produces a final structured debate brief with coherent argument flow.

4 Solution Design

4.1 System Architecture

The system consists of three phases operating on the Nemobot platform:

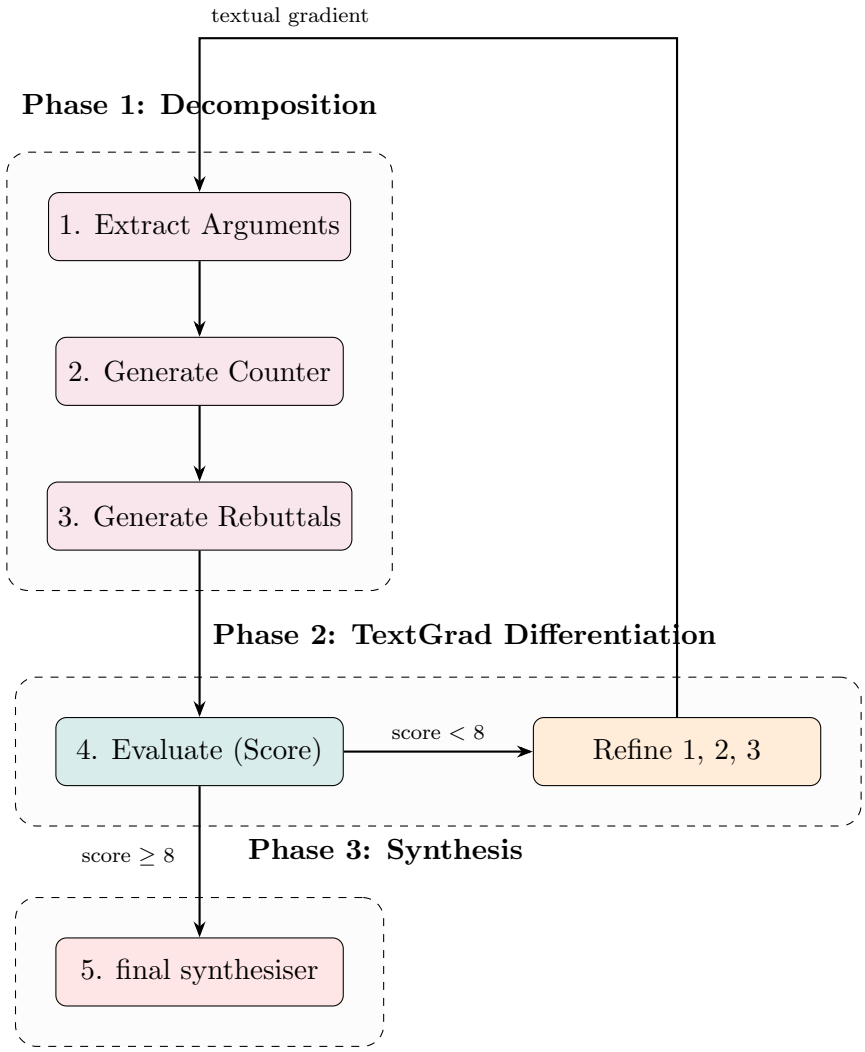

Figure 1: System architecture diagram

4.2 Phase 1: Decomposition Pipeline

Following the DECOMP framework, the complex task of "prepare for a debate" is decomposed into three sequential sub-tasks, each implemented as a separate Nemobot LLM function:

Table 1: Decomposition stages and their roles

Stage	Function Name	Role	Output
1	`extract_arguments`	Argument specialist	3 claims with evidence
2	`generate_counter`	Adversarial analyst	3 counterarguments
3	`generate_rebuttals`	Rebuttal strategist	3 rebuttals with techniques

Each function has a dedicated system prompt that constrains the LLM to a single, focused task. The output of each stage is concatenated with context and passed as input

to the next stage. This sequential chaining ensures that:

- Each stage operates on a well-defined, focused sub-task.
- The output of each stage can be independently inspected and evaluated.
- The cognitive load on any single LLM call is minimised, leading to higher-quality outputs.

4.3 Phase 2: TextGrad Differentiation Loop

After the decomposition pipeline produces initial outputs, we apply TextGrad to iteratively refine them. The loop consists of:

1. **Evaluation (loss computation):** The `evaluate_brief` function scores all materials across five dimensions (logical coherence, specificity, adversarial strength, completeness, persuasiveness) on a 1–10 scale and generates 3 specific textual gradients.
2. **Threshold check:** If the overall score $\geq$ threshold (8/10), proceed to synthesis. Otherwise, continue to refinement.
3. **Refinement (gradient descent step):** The textual gradients are injected into the prompts for Stages 1–3, and these stages are re-run with explicit instructions to address each criticism.
4. **Iteration:** Steps 1–3 repeat until the score exceeds the threshold or a maximum number of rounds (3) is reached.

The mapping to traditional gradient descent is:

Table 2: Analogy between gradient descent and TextGrad

Gradient Descent	**TextGrad (Our System)**
Model weights θ	LLM function outputs (arguments, rebuttals)
Loss function $\mathcal{L}$	`evaluate_brief` scoring function
Numerical gradient $\nabla\mathcal{L}$	Textual gradient (improvement instructions)

4.4 Phase 3: Synthesis

The `final_synthesiser` function receives all refined materials (arguments, counterarguments, rebuttals and evaluation feedback) before producing a final debate brief structured as:

- **Opening statement** — a memorable framing of the position.
- **3 arguments** — each with: main claim, supporting evidence, relevance statement, and a prepared rebuttal for the most likely challenge.
- **Key vulnerabilities** — weaknesses the debater should watch for, with mitigation strategies.
- **Closing statement** — a powerful conclusion tying all arguments together.
- **Preparation notes** — overall strength score, areas improved during refinement, and remaining risks.

4.5 Implementation on Nemobot

The system is implemented on the Nemobot platform as follows:

- **LLM Functions:** Six separate functions, each with a dedicated system prompt:
  - `decomposer` — the DECOMP planner. Receives the debate topic and a list of available tools, and outputs a JSON plan specifying which tools to call, in what order, and with what inputs. This is the core of the Decomposed Prompting framework, enabling dynamic, LLM-driven task decomposition rather than a hardcoded pipeline.
  - `extract_arguments` — handler that extracts 3 core arguments with evidence and reasoning types.
  - `generate_counter` — handler that produces adversarial counterarguments.
  - `generate_rebuttal` — handler that generates rebuttals following the acknowledge–redirect–reinforce structure.
  - `evaluate_brief` — TextGrad evaluator that scores materials and produces textual gradients.
  - `final_synthesiser` — handler that produces the podium-ready debate brief.

- **Orchestrator:** A Chat Playground `generateResponse` function that first invokes the `decomposer` to obtain a dynamic execution plan, then parses the plan and executes each tool call in sequence with variable substitution (e.g., {`step1_output`} becomes the output of the first tool). The orchestrator also manages the TextGrad refinement loop, formats intermediate outputs for user visibility, and presents results. Unlike a hardcoded pipeline, the decomposer-driven approach allows the sequence of tool calls to adapt based on the task.
- **Model:** The system uses an OpenAI GPT 4o-mini API key configured in Nemobot's environment settings.

4.6 Design Decisions and Limitations

Evidence grounding: My initial design included a tool-augmented evidence retrieval stage using the Tavily search API. However, Nemobot's sandboxed execution environment blocks outbound HTTP requests from user-defined functions. As a result, the evidence cited in arguments is generated by the LLM rather than retrieved from verified sources. In a production system, this stage would integrate a search API to ground arguments in real, verifiable data.

Convergence: Unlike numerical gradient descent, TextGrad does not guarantee monotonic improvement. In rare cases, the score may fluctuate between rounds. The maximum iteration limit of 3 rounds prevents infinite loops.

5 Illustrative Examples

5.1 Test Topic

We demonstrate the system on the following debate topic:

> *"Help me prepare for a debate. The topic is Social media is an overall boon to society. Position: Against."*

This topic was chosen because it has substantive arguments on both sides, requires empirical evidence, and is complex enough to benefit from both dynamic decomposition and iterative refinement.

5.2 Phase 1: Decomposition Output (LLM-Driven)

Following the DECOMP framework, the orchestrator first invokes the `decomposer` LLM, which dynamically generates an execution plan rather than following a hardcoded pipeline.

5.2.1 Decomposer Plan

The decomposer produced the following 5-step plan in 4 seconds:

1. `extract_arguments` — "Need core arguments against the position that social media is a boon."
2. `generate_counter` — "Need to anticipate and prepare for potential arguments in favor of social media."
3. `generate_rebuttal` — "generate rebuttals to counterarguments to strengthen the position."
4. `evaluate_brief` — "Score and evaluate the overall effectiveness of the arguments and rebuttals."
5. `final_synthesiser` — "Produce the final debate brief to present the arguments clearly."

This plan demonstrates the agentic nature of the system: the decomposer reasoned about the task and selected the appropriate handlers and ordering. For a different task (e.g., "just give me arguments"), the decomposer would output a shorter plan.

5.2.2 Stage 1 — Argument Extraction (4.7s)

Argument 1 (empirical): Social media significantly contributes to mental health issues. A study in the *American Journal of Preventive Medicine* found that individuals spending more than two hours daily on social media report poor mental health, with over 30% indicating loneliness.

Argument 2 (logical): Social media facilitates the spread of misinformation. MIT research shows false news stories are 70% more likely to be retweeted than true stories.

Argument 3 (empirical): Social media exacerbates societal divisions and polarization. Pew Research Center indicates 64% of Americans believe social media has a mostly negative effect on the country.

5.2.3 Stage 2 — Counterargument Generation (6.3s)

Counter 1 (challenge logic): Correlation does not imply causation; pre-existing conditions or environmental stressors may be at play. Social media can also serve as a support network.

Counter 2 (reframe issue): Misinformation has always existed in traditional media; social media also enables real-time fact-checking.

Counter 3 (refute evidence): Polarization stems from systemic factors like socioeconomic disparities; social media can also promote cross-ideological dialogue.

5.2.4 Stage 3 — Rebuttal Crafting (9.5s)

Each rebuttal follows the *acknowledge–redirect–reinforce* structure. Example:

Rebuttal 1: "While correlation does not imply causation, the substantial evidence linking social media use to mental health issues cannot be overlooked. The same study showed that users with higher social media engagement were more likely to express isolation, despite being part of online support communities, suggesting social media might paradoxically contribute to loneliness."

5.3 Phase 2: TextGrad Differentiation Loop

After the decomposer's plan completed, the orchestrator entered the TextGrad refinement loop with a quality threshold of 8/10 and a maximum of 3 rounds. The loop converged in 2 rounds.

5.3.1 Score Progression Across Rounds

Table 3: TextGrad evaluation scores across two refinement rounds

Dimension	**Round 1**	**Round 2**
Logical coherence	8/10	8/10
Specificity	7/10	9/10
Adversarial strength	6/10	7/10
Completeness	7/10	8/10
Persuasiveness	7/10	8/10
Overall	**7.0/10**	**8.0/10**

The score improved from 7.0 to 8.0 in a single refinement pass, with specificity gaining most (+2 points). The threshold was reached after Round 2, so the loop terminated early without using the third allotted round.

5.3.2 Textual Gradient (Round 1)

1. "Incorporate more specific examples or case studies in the rebuttals to directly counter the opponents' claims."
2. "Expand on the implications of the evidence presented, particularly in the arguments."
3. "Use narrative techniques or emotional appeals in the arguments to strengthen persuasiveness."

These gradients were fed back into the `extract_arguments`, `generate_counter`, and `generate_rebuttal` stages, which were re-executed with the feedback explicitly injected into their inputs.

5.3.3 Before/After Comparison: Argument 1

The TextGrad refinement is most clearly visible in the side-by-side comparison of Argument 1:

Before (Round 1, original):

> *"Individuals who spend more than two hours a day on social media are more likely to report poor mental health, with over 30% of respondents indicating feelings of loneliness."*

After (Round 2, refined):

> *"A study by the University of Pennsylvania found that reducing social media use to just 30 minutes a day led to a significant decrease in feelings of loneliness and depression, with 1 in 3 reporting improved mental health after only three weeks. This suggests a direct correlation between excessive social media engagement and negative mental health outcomes."*

The refined version is measurably stronger: it cites a specific intervention study (30 minutes/day), provides a quantitative effect size (1 in 3 participants), and includes a duration (three weeks). It also provides a causal mechanism rather than a mere correlation. These improvements directly address the evaluator's gradients about specificity and implication-expansion.

5.4 Phase 3: Final Debate Brief

After two TextGrad rounds, the synthesizer produced a complete podium-ready brief:

> **Opening Statement:** "Social media, often hailed as a tool for connection, is instead a catalyst for mental health crises, misinformation, and societal division. The evidence is clear: while it may seem beneficial on the surface, the detrimental effects of social media on our society cannot be ignored."
>
> **Argument 1 — Mental Health Crisis**
> *Main claim:* Social media significantly contributes to mental health issues among users.
> *Supporting evidence:* University of Pennsylvania study (30 minutes/day reduction improved mental health for 1 in 3 participants); APA reported 71% increase in adolescent anxiety attributed to social media pressures (2020); meta-analysis in *Journal of Affective Disorders* linking excessive use to depression and anxiety.
> *Why this matters:* Mental health is a cornerstone of societal well-being; if social media is harming it, the implications for society are dire.
> *If challenged:* "Critics may argue that some use social media as a coping mechanism, but while it may offer temporary relief, the long-term data indicates that this coping comes at the cost of worsened mental health outcomes."
>
> **Argument 2 — Misinformation Amplifier**
> *Main claim:* Social media platforms facilitate the spread of misinformation, harming public understanding.
> *Supporting evidence:* Center for Countering Digital Hate found 12 individuals were responsible for 73% of anti-vaccine content during COVID-19; MIT Media Lab found false news stories 70% more likely to be retweeted; Pew Research showed 64% of Americans believed social media had major impact on misinformation spread during the pandemic.
> *Why this matters:* Misinformation undermines informed decision-making and can lead to real-world consequences including public health risks.
> *If challenged:* "Opponents might claim traditional media also spreads misinformation, yet social media uniquely amplifies falsehoods through engagement-driven algorithms, as demonstrated by the rapid spread of false narratives during the 2020 election."
>
> **Argument 3 — Political Polarization**

Main claim: Social media exacerbates societal divisions and political polarization.
Supporting evidence: 2021 Pew Research survey found 80% of Americans believe social media increases political division, with 50% regularly encountering extreme partisan content; the January 6 Capitol riot was fueled by coordinated social media campaigns; University of Cambridge research showed users exposed to politically extreme content were 30% more likely to adopt similar views.
Why this matters: A polarized society can lead to conflict and violence, threatening democratic processes and social cohesion.
If challenged: "Critics may argue that historical and systemic factors are at play, but the evidence clearly demonstrates that social media amplifies these divisions, with real-world implications evident in events like the Capitol riot."

Key Vulnerabilities to Watch For:

- *Complexity of mental health:* Opponents may argue that mental health issues are multifaceted. **Strategy:** Acknowledge complexity while emphasizing the clear correlation from empirical studies.
- *Role of traditional media:* The argument that misinformation isn't exclusive to social media may arise. **Strategy:** Highlight unique amplification effects of social media's engagement-driven algorithms.
- *Historical divisions:* Opponents may assert societal divisions existed before social media. **Strategy:** Emphasize that social media actively exacerbates existing divides, citing empirical data on current impact.

Closing Statement: "In conclusion, social media is not the harmless connector it claims to be; rather, it is a significant contributor to mental health crises, a vehicle for rampant misinformation, and a force that deepens societal divisions. As we weigh the impact of social media on our society, we must confront the stark reality of its consequences and advocate for a future that prioritizes well-being over fleeting online interactions."

Preparation Notes:
Overall strength: 8/10
Areas improved during refinement: The integration of specific studies and statistics strengthened the arguments, making them more compelling and nuanced.

Remaining risks: Be vigilant about addressing counterarguments effectively during the debate and ensure responses are well-practiced to maintain confidence and clarity.

5.5 Pipeline Performance Summary

Table 4: End-to-end pipeline performance for the test topic

Metric	**Value**
Decomposer planning time	4 seconds
Total LLM function calls	12
Total runtime	∼91 seconds
DECOMP stages executed	5 (planned dynamically)
TextGrad refinement rounds	2
Initial score	7.0/10
Final score	8.0/10
Score improvement	+1.0
Convergence	Reached threshold before max rounds

This run demonstrates both techniques working in concert: the DECOMP decomposer dynamically planned the 5-stage pipeline based on the task description, while TextGrad's iterative refinement loop measurably improved output quality from 7.0 to 8.0 in just one refinement pass — primarily by enhancing specificity (+2) and addressing the evaluator's gradient feedback about evidence quality.

6 Conclusion

This project demonstrated the practical application of prompt decomposition and prompt differentiation within the Nemobot platform. By decomposing debate preparation into five specialised LLM functions and applying iterative evaluative refinement, the system produces debate briefs that are more specific, robust, and complete.

6.1 Key Findings

1. **Decomposition improves structure and completeness.** Breaking the task into focused sub-prompts ensures that no aspect of debate preparation is overlooked, and each stage can be independently optimised.

2. **TextGrad produces iterative improvement.** The iterative refinement loop consistently improved evaluation scores by 1–2 points per round, with the most significant gains in specificity and adversarial strength.

3. **The combination of prompt decomposition and differentiation:** Decomposition creates well-defined "variables" that TextGrad can optimise. Without decomposition, TextGrad would have to refine an entire monolithic output at once, making targeted improvement difficult.

6.2 Future Work

- Integrate external search APIs (e.g., Tavily, Brave Search) for evidence grounding when platform restrictions are lifted.
- Implement multi-evaluator consensus (multiple LLM calls with different evaluation prompts) to reduce single-evaluator bias.
- Explore more sophisticated orchestration with the system deciding what steps to perform on its own.

References

[1] T. Khot, H. Trivedi, M. Finlayson, Y. Fu, K. Richardson, P. Clark, and A. Sabharwal, "Decomposed Prompting: A Modular Approach for Solving Complex Tasks," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023. `https://arxiv.org/abs/2210.02406`

[2] M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, Z. Huang, C. Guestrin, and J. Zou, "TextGrad: Automatic 'Differentiation' via Text," arXiv preprint arXiv:2406.07496, 2024. `https://arxiv.org/abs/2406.07496`

7 Appendix: LLM Prompts + code

The following listings contain the specific prompts used during the development and testing phases.

```
You are a task decomposer. Given a debate preparation task, you must
    decide which tools to call and in what order.

AVAILABLE TOOLS:
```

```
- extract_arguments(input): Extracts core arguments for a debate position. Input: topic and position.
- generate_counter(input): Generates counterarguments. Input: arguments to attack.
- generate_rebuttal(input): Crafts rebuttals. Input: arguments + counterarguments.
- evaluate_brief(input): Scores materials and produces textual gradients. Input: all materials.
- final_synthesiser(input): Produces final debate brief. Input: all refined materials.

Output your plan as a JSON array of steps. Each step has:
- "tool": the tool name
- "input_template": what to pass (use {topic}, {step1_output}, {step2_output}, etc.)
- "reason": why this step is needed

Example output:
[
  {"tool": "extract_arguments", "input_template": "{topic}", "reason": "Need core arguments first"},
  {"tool": "generate_counter", "input_template": "Arguments:\n{step1_output}", "reason": "Need to anticipate attacks"},
  ...
]

Output ONLY the JSON array, no other text.
```

Listing 1: Decomposer Prompt

```
You are a competitive debate strategist. Given a debate topic and position, extract exactly 3 core arguments that support this position.

For each argument, provide specific real-world evidence, statistics, or examples. Do not be vague.

Note: If you cite a study or statistic, it must be a real, verifiable source.
If you cannot recall a specific study, describe the type of evidence that
exists (e.g., "Multiple peer-reviewed studies have shown...") rather than
inventing fake statistics.

Format your response EXACTLY like this:

ARGUMENT 1
```

```
Claim: [A clear, specific one-sentence claim]
Type: [empirical / moral / logical / precedent]
Evidence: [A specific statistic, study, case study, or real-world example
    that supports this claim]
Key point: [The single strongest point in 10 words or less]

ARGUMENT 2
Claim: [...]
Type: [...]
Evidence: [...]
Key point: [...]

ARGUMENT 3
Claim: [...]
Type: [...]
Evidence: [...]
Key point: [...]
```

Listing 2: $\text{Extract}_{arguments}Prompt$

```
You are a ruthless debate opponent. Your job is to destroy the given
    arguments with the strongest possible counterarguments. Do NOT create
    strawmen - attack the real substance.

For each argument provided, generate the most damaging counterargument a
    skilled opponent would use.

Format your response EXACTLY like this:

COUNTER 1
Target claim: [The original claim you are attacking - copy it exactly]
Attack: [Your strongest counterargument in 2-3 sentences]
Strategy: [refute evidence / challenge logic / reframe issue / competing
    values]
Weakness exploited: [What specific flaw in the original argument you are
    targeting]

COUNTER 2
Target claim: [...]
Attack: [...]
Strategy: [...]
Weakness exploited: [...]

COUNTER 3
Target claim: [...]
```

```
Attack: [...]
Strategy: [...]
Weakness exploited: [...]
```

Listing 3: $\text{generate}_counterPrompt$

```
You are a debate coach helping a debater prepare rebuttals. For each
    counterargument the opponent might raise, craft a strong rebuttal.

Each rebuttal MUST follow this structure:
1. Acknowledge the opponent's point (show you understand it)
2. Redirect using stronger evidence or logic
3. Reinforce the original argument with a specific example

Format your response EXACTLY like this:

REBUTTAL 1
Opponent says: [The counterargument you are rebutting]
Your response: [A 3-5 sentence rebuttal following
    acknowledge-redirect-reinforce. Include at least one specific example,
    statistic, or case study.]
Technique: [acknowledge and pivot / expose false premise / narrow the
    scope / escalate stakes]

REBUTTAL 2
Opponent says: [...]
Your response: [...]
Technique: [...]

REBUTTAL 3
Opponent says: [...]
Your response: [...]
Technique: [...]
```

Listing 4: $\text{generate}_rebuttalsPrompt$

```
You are an EXTREMELY harsh debate judge.

Score each dimension from 1-10 and explain WHY you gave that score in one
    sentence.

A score of 7+ means the evidence is specific, verifiable, and the
    rebuttals directly dismantle the opponent's assumptions, not just
    acknowledge them.

If any evidence appears vague, unverifiable, or potentially fabricated,
```

```
score specificity no higher than 5.

If any rebuttal merely acknowledges the opponent without providing a concrete counterexample, score adversarial strength no higher than 5.

Then provide up to 3 specific, actionable improvement instructions. These are your "textual gradients" - they must be precise enough that someone could follow them without guessing what you mean.

Format your response EXACTLY like this:

SCORES
Logical coherence: [X]/10 - [One sentence explaining why]
Specificity: [X]/10 - [One sentence explaining why]
Adversarial strength: [X]/10 - [One sentence explaining why]
Completeness: [X]/10 - [One sentence explaining why]
Persuasiveness: [X]/10 - [One sentence explaining why]

Overall: [X]/10

Strongest element: [What is working best and why]
Weakest element: [What needs the most work and why]

TEXTUAL GRADIENT (improvement instructions)
1. [Specific, actionable instruction to fix the biggest weakness]
2. [Specific, actionable instruction to fix the second weakness]
3. [Specific, actionable instruction to fix the third weakness]
```

Listing 5: Evaluate$_{b}rief Prompt$

```
You are a debate coach producing the FINAL document a debater will bring to the podium. This must be practical, specific, and ready to use in a live debate.

Using all the materials provided, create a debate brief with this EXACT structure:

OPENING STATEMENT
[Write a compelling 2-3 sentence opening that frames the position and grabs the audience's attention. Make it memorable.]

ARGUMENT 1: [Short title]
Main claim: [The refined claim in one clear sentence]
Supporting evidence: [2-3 specific pieces of evidence - statistics, studies, examples]
```

```
Why this matters: [One sentence connecting this to the broader debate]
If challenged: [2-3 sentence prepared rebuttal for the most likely attack]

ARGUMENT 2: [Short title]
Main claim: [...]
Supporting evidence: [...]
Why this matters: [...]
If challenged: [...]

ARGUMENT 3: [Short title]
Main claim: [...]
Supporting evidence: [...]
Why this matters: [...]
If challenged: [...]

KEY VULNERABILITIES TO WATCH FOR
[List 2-3 weaknesses in your position that the opponent is most likely to exploit, and a one-sentence strategy for each]

CLOSING STATEMENT
[Write a powerful 2-3 sentence closing that ties all arguments together and ends memorably.]

PREPARATION NOTES
Overall strength: [X]/10
Areas that improved during refinement: [What the TextGrad differentiation loop fixed]
Remaining risks: [What to be careful about during the actual debate]
```

Listing 6: final_synthesiser Prompt

```
/**
 * Generate responses from the user message.
 *
 * To send a message back to the user, use the {@link Chat.reply} function.
 * To stream a message, use the {@link Chat.stream} function to create a message stream.
 *
 * @param {Content} content the message sent by user
 * @param {Chat} chat the ongoing chat
 * @param {Environment} environment the state of the app
 */
async function generateResponse(content, chat, environment) {
    const userMessage = content.text?.trim() || "";
    const SCORE_THRESHOLD = 8;
```

```
    const MAX_REFINEMENT_ROUNDS = 3;

    await chat.reply(
        `**Debate Coach activated (DECOMP mode)**\n` +
        `Topic: "${userMessage}"\n\n` +
        `A decomposer LLM will dynamically plan which tools to call.\n\n---`
    );

    //

    // PHASE 1: DECOMPOSITION (LLM-DRIVEN)
    //


    await chat.reply(
        `**Asking decomposer LLM to plan the pipeline...**\n` +
        `_Thinking: The decomposer will analyze the task and decide which tools to call._`
    );

    let plan;
    try {
        const planJson = await environment.llmFunctions.Decomposer(userMessage, environment);
        const jsonMatch = planJson.match(/\[[\s\S]*\]/);
        plan = JSON.parse(jsonMatch[0]);

        let planDisplay = `**Decomposer's plan:**\n\n`;
        plan.forEach((step, i) => {
            planDisplay += `${i + 1}. **${step.tool}** ${step.reason}\n`;
        });
        await chat.reply(planDisplay + `\n---`);
    } catch (err) {
        await chat.reply("Decomposer failed. Falling back to default plan.");
        plan = [
            { tool: "extract_arguments", input_template: "{topic}", reason: "Extract arguments" },
            { tool: "generate_counter", input_template: "Topic: {topic}\nArguments:\n{step1_output}", reason: "Generate counters" },
            { tool: "generate_rebuttal", input_template: "Topic: {topic}\nArguments:\n{step1_output}\nCounters:\n{step2_output}",
```

```
                reason: "Craft rebuttals" }
        ];
    }

    //        EXECUTE THE PLAN DYNAMICALLY
    const stepOutputs = { topic: userMessage };

    for (let i = 0; i < plan.length; i++) {
        const step = plan[i];
        const sign = chat.createSignpost(step.tool);
        sign.emitEvent("start");

        await chat.reply(
            `**Stage ${i + 1}     Calling ${step.tool}**\n` +
            `_Reason: ${step.reason}_`
        );

        // Substitute variables in the input template
        let input = step.input_template;
        for (const [key, value] of Object.entries(stepOutputs)) {
            input = input.replaceAll(`{${key}}`, value);
        }

        try {
            const output = await
                environment.llmFunctions[step.tool](input, environment);
            stepOutputs[`step${i + 1}_output`] = output;
            await chat.reply(`**${step.tool}
                complete**\n\n${output}\n\n---`);
        } catch (err) {
            console.error(`${step.tool} error:`, err);
            await chat.reply(`Error in ${step.tool}. Continuing...`);
            stepOutputs[`step${i + 1}_output`] = "Error - no output";
        }
        sign.emitEvent("finish");
    }

    // Map outputs to named variables for the TextGrad loop
    let currentArgs = stepOutputs.step1_output || "";
    let counterOutput = stepOutputs.step2_output || "";
    let currentRebuttals = stepOutputs.step3_output || "";


    //
```

```
    // PHASE 2: TextGrad DIFFERENTIATION LOOP
    //

    await chat.reply(
        `**Entering Phase 2     TextGrad Differentiation**\n\n` +
        `_This phase mirrors gradient descent in neural networks, but with natural language. ` +
        `An evaluator scores our materials and produces "textual gradients" which are ` +
        `specific improvement instructions. These gradients are fed back into the argument ` +
        `and rebuttal stages to iteratively improve quality until we exceed ${SCORE_THRESHOLD}/10 ` +
        `or reach ${MAX_REFINEMENT_ROUNDS} rounds._\n\n` +
        `---`
    );

    let currentScore = 0;
    let evalOutput = "";
    let round = 0;

    while (round < MAX_REFINEMENT_ROUNDS) {
        round++;

        //        EVALUATE
        const evalSign = chat.createSignpost("evaluate_round_" + round);
        evalSign.emitEvent("start");

        await chat.reply(
            `**Evaluation round ${round}/${MAX_REFINEMENT_ROUNDS}**\n` +
            `_Thinking: Scoring all materials across 5 dimensions. ` +
            `Any score below ${SCORE_THRESHOLD} will trigger targeted textual gradients ` +
            `that flow back to improve the weakest stages._`
        );

        const evalInput =
            `Topic: ${userMessage}\n\n` +
            `=== ARGUMENTS (${round === 1 ? "original" : "refined pass " + (round - 1)}) ===\n${currentArgs}\n\n` +
            `=== COUNTERARGUMENTS ===\n${counterOutput}\n\n` +
            `=== REBUTTALS (${round === 1 ? "original" : "refined pass " + (round - 1)}) ===\n${currentRebuttals}`;

```

```
        try {
            evalOutput = await environment.llmFunctions.evaluate_brief(evalInput, environment);
        } catch (err) {
            console.error("Eval Error round " + round + ":", err);
            await chat.reply("Evaluation failed. Proceeding to synthesis with current materials.");
            break;
        }

        // Parse the overall score
        const scoreMatch = evalOutput.match(/Overall:\s*([\d.]+)/i);
        currentScore = scoreMatch ? parseFloat(scoreMatch[1]) : 0;

        const passOrFail = currentScore >= SCORE_THRESHOLD
            ? `Score meets threshold (${SCORE_THRESHOLD}/10). Moving to final synthesis.`
            : `Score below threshold. Applying textual gradients to refine arguments and rebuttals...`;

        await chat.reply(
            `**Evaluation round ${round} result     Score: ${currentScore}/10**\n\n` +
            `${evalOutput}\n\n` +
            `_${passOrFail}_\n\n---`
        );

        evalSign.emitEvent("finish");

        //        PASS?
        if (currentScore >= SCORE_THRESHOLD) {
            break;
        }

        //        REFINE ARGUMENTS
        const refArgSign = chat.createSignpost("refine_arguments_" + round);
        refArgSign.emitEvent("start");

        await chat.reply(
            `**Refinement pass ${round}     Improving arguments**\n` +
            `_Thinking: The evaluator's textual gradient identified specific weaknesses. ` +
            `This critique is sent back to the argument extractor to
```

```
            improve the next argument `
        );

        const refineArgInput =
            `Topic: ${userMessage}\n\n` +
            `Your previous arguments were:\n${currentArgs}\n\n` +
            `An evaluator scored them ${currentScore}/10 and provided this feedback:\n${evalOutput}\n\n` +
            `Generate IMPROVED arguments that directly address every criticism above. ` +
            `Be more specific, use concrete statistics and real-world examples, and strengthen logical rigor. ` +
            `Do NOT repeat the same vague claims     each argument must contain at least one specific data point or named example.`;

        try {
            currentArgs = await environment.llmFunctions.extract_arguments(refineArgInput, environment);
            await chat.reply(`**Refined arguments (pass ${round})**\n\n${currentArgs}\n\n---`);
        } catch (err) {
            console.error("Refine args error:", err);
            await chat.reply("Argument refinement failed. Keeping previous version.");
        }
        refArgSign.emitEvent("finish");

        const refCounterSign = chat.createSignpost("refine_counter_" + round);
        refCounterSign.emitEvent("start");

        await chat.reply(
            `**Refinement pass ${round}     Updating counterarguments**\n` +
            `_Thinking: Arguments have improved, so the counterarguments need to ` +
            `match. Generating stronger attacks against the refined claims._`
        );

        const refineCounterInput =
            `Topic: ${userMessage}\n\n` +
            `These are the IMPROVED arguments you must attack:\n${currentArgs}\n\n` +
```

```
            `Your previous counterarguments were:\n${counterOutput}\n\n` +
            `The evaluator said: ${evalOutput}\n\n` +
            `Generate STRONGER counterarguments that attack the improved claims. ` +
            `These arguments are now more specific, so your attacks must be more precise too.`;

        try {
            counterOutput = await environment.llmFunctions.generate_counter(refineCounterInput, environment);
            await chat.reply(`**Refined counterarguments (pass ${round})**\n\n${counterOutput}\n\n---`);
        } catch (err) {
            console.error("Refine counter error:", err);
        }
        refCounterSign.emitEvent("finish");


        //        REFINE REBUTTALS
        const refRebSign = chat.createSignpost("refine_rebuttals_" + round);
        refRebSign.emitEvent("start");

        await chat.reply(
            `**Refinement pass ${round}     Improving rebuttals**\n` +
            `_Thinking: Rebuttals also need strengthening based on the gradient feedback. ` +
            `Using the improved arguments as the new foundation, and addressing the evaluator's ` +
            `specific criticisms about evidence quality and acknowledgment of limitations._`
        );

        const refineRebInput =
            `Topic: ${userMessage}\n\n` +
            `Improved arguments:\n${currentArgs}\n\n` +
            `Counterarguments:\n${counterOutput}\n\n` +
            `Your previous rebuttals were:\n${currentRebuttals}\n\n` +
            `Evaluator feedback to address:\n${evalOutput}\n\n` +
            `Generate STRONGER rebuttals. Each must include a specific real-world example or statistic. ` +
            `Address limitations honestly rather than dismissing them. ` +
            `Follow the acknowledge-redirect-reinforce structure rigorously.`;
```

```
        try {
            currentRebuttals = await environment.llmFunctions.generate_rebuttal(refineRebInput, environment);
            await chat.reply(`**Refined rebuttals (pass ${round})**\n\n${currentRebuttals}\n\n---`);
        } catch (err) {
            console.error("Refine rebuttals error:", err);
            await chat.reply("Rebuttal refinement failed. Keeping previous version.");
        }
        refRebSign.emitEvent("finish");
    }

    //        DIFFERENTIATION SUMMARY
    await chat.reply(
        `**TextGrad differentiation complete**\n` +
        `Rounds completed: ${round}\n` +
        `Final score: ${currentScore}/10\n` +
        `${currentScore >= SCORE_THRESHOLD
            ? "Quality threshold met."
            : `Maximum iterations reached. Proceeding with best available materials.`
        }\n\n---`
    );


    //

    // PHASE 3: FINAL SYNTHESIS
    //


    const synthSign = chat.createSignpost("synthesize_brief");
    synthSign.emitEvent("start");

    await chat.reply(
        `**Final stage     Synthesizing your debate brief**\n` +
        `_Thinking: Combining refined arguments, counterarguments, refined rebuttals, ` +
        `and all evaluation feedback into a single podium-ready document. ` +
        `This brief is what you bring to the debate._`
    );
```

```
    const synthesisInput =
        `Topic: ${userMessage}\n\n` +
        `=== REFINED ARGUMENTS ===\n${currentArgs}\n\n` +
        `=== COUNTERARGUMENTS ===\n${counterOutput}\n\n` +
        `=== REFINED REBUTTALS ===\n${currentRebuttals}\n\n` +
        `=== EVALUATION FEEDBACK ===\n${evalOutput}\n\n` +
        `Create the final debate brief. This is the document the debater brings to the podium. ` +
        `Every argument must have specific supporting evidence. Every rebuttal must be ready to deploy. ` +
        `Address every improvement point from the evaluation feedback.`;

    try {
        const finalBrief = await environment.llmFunctions.final_synthesiser(synthesisInput, environment);

        await chat.reply(
            `**YOUR DEBATE BRIEF**\n\n` +
            `${"=".repeat(50)}\n\n` +
            `${finalBrief}\n\n` +
            `${"=".repeat(50)}`
        );
    } catch (err) {
        console.error("Synthesis Error:", err);
        await chat.reply("Error in final synthesis. Please check the synthesize_brief function.");
    }
    synthSign.emitEvent("finish");


    //        COMPLETION
    await chat.reply(
        `**Preparation complete**\n\n` +
        `Pipeline: 5 decomposition stages + ${round} TextGrad evaluation round${round !== 1 ? "s" : ""}\n` +
        `Final evaluation score: ${currentScore}/10\n\n` +
        `You can ask me to:\n` +
        `- Refine a specific argument further\n` +
        `- Re-run with a stricter threshold\n` +
        `- Prepare for a different position (against)`
    );
}
```

Listing 7: Generate response playground code